Guía técnica de la demostración

Este documento describe, paso a paso, lo que se ejecuta en la demostración del proyecto: qué función se llama en cada momento, qué hace cada una y qué parámetros admite. Es la contrapartida escrita de los dos recorridos que se enseñan en directo, el de terminal (`scripts/sesion.py`) y el de la interfaz web (`src/app/serve.py`).

El proyecto replica *Feature Visualization* (Olah, Mordvintsev y Schubert, 2017) sobre un ResNet-18 preentrenado en Pl@ntNet-300K. Conviene subrayar algo desde el principio, porque es el centro del método: **los pesos del modelo no se entrenan ni se modifican en ningún momento**. Se cargan congelados (`requires_grad_(False)`) y lo único que recibe gradiente, y por tanto lo único que se optimiza, es la imagen de entrada.

1. Las funciones, en el orden en que se ejecutan

Todas viven en `src/interpretability/api.py`, que es la fachada del proyecto. La interfaz web y el recorrido de terminal llaman exactamente a las mismas.

Llamada	Qué hace	Qué devuelve
`cargar_modelo()`	Lee el checkpoint y congela los pesos	`nn.Module` (11.731.065 parámetros)
`predecir(foto, top_k)`	Pasada hacia delante y softmax	`list[Prediccion]`
`listar_neuronas(foto, capas, top_k)`	Mide qué canales enciende la foto	`list[Activacion]`
`mapa_activacion(foto, neurona)`	Dónde se enciende dentro de la foto	tensor `(224, 224)`
`superponer_mapa(foto, mapa, mezcla)`	Pinta el mapa sobre la foto	tensor `(3, 224, 224)`
`generar_imagen(neurona, tipo, pasos)`	La optimización propiamente dicha	tensor `(1, 3, 224, 224)`

`cargar_modelo()` construye el ResNet-18, carga `models/resnet18_weights_best_acc.tar` y pone el modelo en modo evaluación con los gradientes desactivados. El resultado se cachea, de modo que los pasos siguientes lo reutilizan sin releer el fichero.

`predecir()` normaliza la foto con las estadísticas de ImageNet, la pasa por el modelo y aplica softmax. Cada `Prediccion` lleva el índice de clase, la probabilidad, el identificador de especie de Pl@ntNet y el nombre científico. La traducción de índice a especie se deriva del orden alfabético de los directorios del conjunto de datos, porque el mapa no se publica con él.

`listar_neuronas()` engancha un *hook* a la capa indicada, pasa la foto y promedia espacialmente cada canal. Ordena de mayor a menor y devuelve los `top_k` **por capa**. Cada `Activacion` guarda la capa, el índice de canal y el valor medio. Ese valor solo es comparable dentro de una misma capa: `layer1` activa de media unas cuatro veces más que `layer3`.

`mapa_activacion()` devuelve la rejilla espacial de ese canal *antes* de promediarla, reescalada al tamaño de la foto. Es la versión espacial del número que produjo el ranking, y responde a dónde mira la neurona. **`superponer_mapa()`** la pinta encima de la foto con un peso configurable (`mezcla`, 0,55 por defecto).

`generar_imagen()` es la que importa. Parte de ruido y busca la imagen que más activa esa neurona. Acepta una `Activacion` o varias; si recibe varias las maximiza a la vez, que es el objetivo *jointly optimized* del artículo.

2. Qué ocurre dentro de `generar_imagen()`

La función delega en `maximizar_activacion()` (`src/interpretability/feature_viz.py`), cuyo bucle repite estos pasos en cada iteración:

1. **Parametrización.** `construir_parametrizacion()` produce la imagen a partir de los parámetros que se están optimizando. Con `fourier` esos parámetros son un espectro escalado por `1/f**decay` más una decorrelación de color y una sigmoide: es el *preconditioning* del artículo. Con `pixel` se optimizan los píxeles directamente.
2. **Robustez a transformaciones.** `aplicar_robustez()` encadena relleno, desplazamiento aleatorio, escalado, rotación, un segundo desplazamiento y recorte. Sin este paso la optimización converge a ruido adversarial.
3. **Normalización** con la media y desviación del bloque `normalizacion`.
4. **Pasada hacia delante** por el modelo congelado, capturando la capa objetivo con un *hook*.
5. **Objetivo.** El escalar que se quiere maximizar, según la clase elegida en `src/interpretability/objetivos.py`.

6. **Pérdida.** `-activacion`, más los penalizadores de frecuencia activos (`_norma_l1`, `_variacion_total`, `_penalizacion_desenfoque`) y, si el lote es mayor que uno, `alineacion` y `diversidad`.
7. **`backward()` y `step()`.** El gradiente llega hasta los parámetros de la imagen; Adam los actualiza. Los pesos del modelo quedan intactos.

Si en los primeros pasos el gradiente es exactamente cero, `_comprobar_gradiente()` lanza `UnidadMuerta`. Es el caso de una unidad ReLU muerta: no da error por sí sola, simplemente nunca aprendería nada.

3. Objetivos disponibles

Son los de la sección *Optimization Objectives* del artículo, más los de *The Interaction between Neurons*. Se eligen con `objetivo.tipo` en el fichero de configuración, o con el selector *Objetivo* de la interfaz.

Tipo	Expresión	Notas
`canal`	`layer_n[:, :, z]`	Media del canal en el espacio. El de por defecto
`neurona`	`layer_n[x, y, z]`	Una sola posición del canal
`capa`	`layer_n[:, :, :]²`	DeepDream; ignora el canal
`logit`	`pre_softmax[k]`	Solo con `capa: fc`
`probabilidad`	`softmax[k]`	Solo con `capa: fc`; también hunde las demás clases
`direccion`	`v · layer_n[x, y, :]`	`v` aleatorio de norma 1
`suma`	varios canales a la vez	*Jointly optimized*
`interpolacion`	de un canal a otro	Necesita `imagen.lote >= 2`

En la interfaz se exponen los cinco primeros. Los tres últimos se usan desde `configs/feature_viz.yaml` o desde las funciones `generar_direccion()`, `generar_diversas()` y `generar_interpolacion()` de la fachada.

4. Parámetros configurables

Ningún hiperparámetro está escrito a fuego en el código: todos viven en `configs/feature_viz.yaml`.

Bloque	Claves	Valor por defecto
`experimento`	`nombre`, `seed`	`feature-viz`, 42
`modelo`	`arquitectura`, `checkpoint`, `n_clases`	resnet18, 1081 clases
`objetivo`	`tipo`, `capa`, `canal`, `x`, `y`, `direccion`, `canales`	`canal`, `fc`, 0
`imagen`	`tamano`, `parametrizacion`, `lote`, `compartida`, `decay_power`, `sd_inicial`	224, `fourier`, 1, 1.0, 0.01
`optimizacion`	`learning_rate`, `pasos`	0,05 y 256
`regularizacion`	`l1`, `tv`, `blur`, `l2`, `alineacion`, `diversidad`	todos a 0 salvo `alineacion.peso` (100)
`robustez`	`activa`, `pad`, `jitter`, `jitter_final`, `escalas`, `rotaciones`	activa; 16, 16, 8; ±5 % y ±5°
`normalizacion`	`media`, `desviacion`	estadísticas de ImageNet
`salida`	`directorio`	`outputs/feature_viz`
`wandb`	`proyecto`, `entidad`, `modo`, `cada_imagen`	`tiiac-2026`, `online`, 32

Dos advertencias que el propio fichero documenta. Los coeficientes del artículo (L1 −0,05, TV −0,25, *blur* −1) **no se pueden copiar tal cual**: allí van sobre sumas de 150.528 elementos y aquí los términos se promedian, y además allí se maximiza y se restan mientras que aquí se minimiza y se suman. Y los valores de robustez son los de la nota al pie 9 del artículo, que no coinciden con los `standard_transforms` de *lucid*.

Los nueve mandos de la interfaz

Son los de las figuras interactivas del artículo. Cada uno sobreescribe la clave correspondiente del fichero de configuración sin modificarlo.

Mando	Rango	Por defecto	Efecto
Pasos de optimización	32–1024	256	Unos 70 ms por paso en CPU
Learning rate	0,005–0,5	0,05	Adam sobre el espectro
L1	0–8	0	Penaliza la media de `abs(imagen)`
Variación total	0–20	0	El más efectivo: baja la aspereza de 0,16 a 0,04
Desenfoque	0–200	0	Distancia a la imagen desenfocada
Robustez activada	sí/no	sí	Apagarla sube la activación y empeora la imagen
Jitter	0–32 px	16	Desplazamiento aleatorio
Rotación máxima	0–40°	5	Genera el rango `-r…r`
Amplitud de escala	0–0,4	0,05	Genera cinco escalas alrededor de 1
Espacio de Fourier	sí/no	sí	Apagarlo converge a ruido adversarial

Las dos casillas de robustez y *preconditioning* son las que mejor ilustran el argumento central del artículo: al apagarlas la activación final sube, pero la imagen deja de parecerse a nada reconocible.

A esos mandos se añade el selector de capa (`layer1.1.conv2` , `layer2.1.conv2` , `layer3.1.conv2` , `layer4.1.conv2` y `fc` , todas previas a la ReLU para evitar unidades muertas) y el interruptor *Registrar en W&B*, apagado por defecto para que la aplicación no abra conexiones de red sin que se le pida.

5. Lo que se registra en Weights & Biases

Cada ejecución con registro activado crea un run cuyo nombre identifica la unidad, por ejemplo `layer4_1_conv2_canal_362` . En cada paso se envían:

- `viz/activacion` : el valor que se está maximizando
- `viz/perdida` : la activación negada más los penalizadores
- `viz/aspereza` : la variación total de la imagen, como medida de ruido
- `viz/alineacion` y `viz/diversidad` , solo si están activas
- `viz/imagen` : la imagen en curso, cada `wandb.cada_imagen` pasos (32 por defecto), que reproduce la figura *Step 1 / 32 / 128 / 256* del artículo

6. Reproducir la demostración

```
python scripts/sesion.py --capa layer4.1.conv2    # recorrido paso a paso
python scripts/demo.py  --capa layer4.1.conv2     # lo mismo, de una vez
streamlit run src/app/serve.py                    # la interfaz web
```

Para que los pasos queden registrados en Weights & Biases hay que exportar `WANDB_API_KEY` antes de ejecutarlos. Sin esa variable todo funciona igual, pero sin registrar.